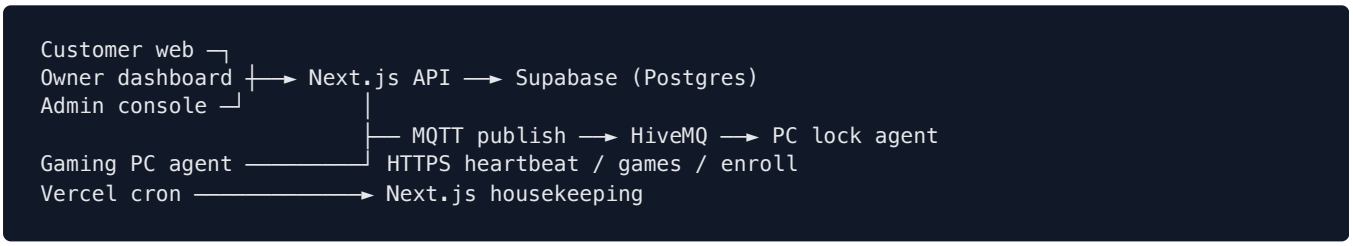
BookMyGame Architecture Review

Senior-engineer reverse-engineering of the café booking platform and PC lock agent

PlayTime Gaming Cafe · bookmygame.co.in · 22 August 2026
Scope: understand architecture and data flow; identify risks; recommend refactors. Product behaviour was not changed.

> This is a **Next.js (App Router) café booking platform** plus a **Windows kiosk lock agent**. Customers book machines; owners run the café; a PC agent locks and unlocks hardware over MQTT. The site is serverless on Vercel; Postgres is Supabase.

1. Architecture as it actually works



Three actors, three auth models

Actor	Proof	Used for
Customer	Supabase JWT (`Authorization: Bearer`)	Book, pay, profile
Owner / admin	HMAC cookie (`owner_session` / `admin_session`)	Dashboards, mutations
Gaming PC	Shared `STATION_HEARTBEAT_TOKEN` + enrolled station row	Heartbeat, games, walk-in play

Core data flow (booking → machine)

1. Customer (or walk-in / scan) creates a `bookings` + `booking_items` row. Price is recalculated server-side from `console_pricing` / `station_pricing`.
2. Stations are reserved in item titles / `station_names`.
3. `syncStationsForBooking` computes remaining seconds in IST and publishes MQTT `unlock` / `lock`.
4. Agent receives the command, starts a local countdown, shows the game menu, heartbeats HTTP so the dashboard can show online/locked.
5. Housekeeping cron completes ended bookings, expires memberships, purges unlock tokens.

Main modules

- `src/app/api/bookings` — customer checkout
- `src/app/api/owner/*` — fat owner BFF (`/data` is the dashboard dump)
- `src/app/api/stations/*` — agent-facing

- `src/lib/stationSync.ts` + `stationCommands.ts` — booking ↔ lock
- `src/lib/availabilityService.ts` — occupancy (client fetch + JS overlap)
- `pc-lock-agent/` — WinForms kiosk (lock screen, menu, MQTT, game catalog)

That split is sound: **the website is the source of truth; the agent is an actuator**.

---

2. What's wrong

Bad architecture

1. **Route handlers are the domain layer.** Pricing, reservation, MQTT, loyalty, and IST time live inside `route.ts` files. There is no booking/session service. Adding "unlock at start time" means hunting ~87 routes.
2. **Owner dashboard is a god object.** `OwnerDashboardContext` + `useOwnerData` load a truncated booking dump (7 days / 300 rows, or 90 days / 500) and compute revenue in the browser. Reports have a second query path. Totals can disagree.
3. **MQTT from serverless.** Every lock/unlock opens a new broker TCP connection, publishes, disconnects. No subscriber on the web side. Status is HTTP polling, not the broker. Fine for a few PCs; fragile at dozens of cafés.
4. **One shared agent token for all cafés.** Enrollment proves "an agent," not "this café's agent." `requireKnownStation` is the only tenancy check. Steal one token, name another café's station.
5. **Admin password in source** (`adminLoginAccount.ts`). This is not an auth design; it is a backdoor in git.
6. **Session secrets fall back to the Supabase key.** If `OWNER_SESSION_SECRET` is unset, the service role (or anon) key signs cookies.
7. **Time stored as strings** (`"6:00 PM"` + date, no timestamptz). Availability, lock remaining, and "is it started yet?" all re-parse IST in JS. This has already caused AM/PM bugs.

Duplicate logic

- IST "today" was copied in owner data, reports, membership checkout, station assignment, owner utils.
- HMAC cookie sign/verify was copied between owner and admin.
- `convertTo12Hour` / `convertTo24Hour` existed in `timeUtils` and owner utils.
- Owner vs admin lookup clients are the same fetch wrapper with different URLs (left as-is; different shapes).

Performance bottlenecks

- `/api/owner/data` is a multi-table join dump on tab switch / 60s refresh.
- Availability loads all of that day's bookings then overlaps in the browser.
- Reports already paginate (`lib/db/pagination`); the live dashboard does not.
- MQTT connect-per-command adds ~100–500ms and broker connection churn.

Scalability risks

- MQTT topics are global (`cafe/station/pc-01/command`) — station names are not café-scoped in the topic.

- Agent catalog/heartbeat is one token + polling, not per-café credentials.
- Hard caps (`FULL_BOOKING_LIMIT = 500`) silently drop history as a café grows.
- No queue: booking write and MQTT publish are the same request. Broker slowness does not fail the booking (good), but there is no retry.

Maintainability

- `any` in owner data/hooks.
- Fallback SQL when columns are missing (`updated_at`, `station_names`) — migrations and runtime have diverged.
- Owner context is thousands of lines of mixed UI + policy.
- README is still the Next.js starter; operational knowledge lives in comments.

---

3. Clean target architecture

```
src/
  domain/
    booking/        create, price, reserve, cancel
    session/        remaining time, lock/unlock policy
    availability/   server-side occupancy
    time/           IST instants (started as indiaTime.ts)
  infra/
    mqtt/           publish with retry
    supabase/
    cookies/        signedCookie.ts
  app/api/          thin: auth → domain → JSON
```

Refactoring priority (no product-behaviour change)

1. Move remaining time + lock policy out of `stationSync` into a pure function + tests (IST, pending, future start).
2. Compute availability on the server from occupied station IDs, not client overlap on full rows.
3. Split owner `/data` into summary / live stations / paged bookings.
4. Per-café agent credentials; café id in MQTT topic.
5. Remove hardcoded admin login; dedicated session secrets with no key fallback.
6. Store session start/end as `timestamptz`.

---

4. Quality upgrades already applied

These preserve product behaviour. Typecheck was clean.

Change	Why
`src/lib/indiaTime.ts`	One IST calendar/clock
Call sites import it (booking filters re-export)	Stops “today” drifting between screens

Change	Why
`src/lib/signedCookie.ts`	One HMAC + cookie-flag implementation
Owner/admin auth use it	Same cookies, less copy-paste
Owner utils re-export time converters from `timeUtils`	One parser
MQTT `clientId` uses `randomUUID()`	Still unique, not Math.random
Station bearer compare is timing-safe	Same accept/reject, harder to probe

The dashboard, booking POST, and lock agent were **not** rewritten — that would be a behaviour risk, not a quality pass.

> **Highest-risk leftover (operations, not “code style”):** hardcoded admin credentials in git; session secret fallback to the database key; one heartbeat token for every café PC. Those should be env-only before more cafés go live.

---

5. Recommended next slice

Extract **session remaining-time + lock decision** from `stationSync.ts` into a tested module, then point every unlock path at it (booking, play-request, owner control, cron). Still no UX change.

Document generated from the in-repo architecture review. Internal implementation notes and credentials are omitted on purpose.